MP2 — Selecting Reliable LLM Reasoning

DASE7506 Advanced Machine Learning Methods — Fall 2026
Official test score: `{"accuracy": 84.36830835117773, "correct": 394, "n": 467}`

Project Overview

A language model (Qwen3-0.6B) produces four candidate solutions to each GSM8K math problem. They can all sound convincing, show calculations, and still disagree on the answer. The task: build a program that reads the question, the four candidate texts, and their fixed 1024-d embeddings, then selects **one candidate index (0–3) per question**. The scorer checks only the **final numeric answer** (including equivalent fractions/decimals), not reasoning validity. Multiple candidates can be correct, and any correct choice earns the point.

Data splits: 4,755 train / 798 validation / 467 test questions.

Constraints: CPU inference ≤30 s (4 threads), ≤16 MiB inference assets, fixed seed 7506, no external data, no answer lookups, frozen method must generalise to unseen audit bank.

---

1. Method

1.1 Evidence groups (1,132 features)

All features vary **within a group** (the four candidates of one question); group-level constants carry no decision information and are excluded. Nine hand-crafted groups plus one dense embedding block:

Group	Features	Description
A	12	Vote structure over parsed final answers (counts, ratios, margins, singletons, ranks, tie-breaks).
B	8	Embedding geometry: norm rel., distance to centroid, cosine to centroid, pairwise cosine stats.
S	4	Answer-cluster coherence: cosine to same/diff-answer centroids, margin, has-support.
C	28	Text surface (absolute): length, counts, formatting markers, step keywords, hesitation/conclusion words, numeric density, answer properties.
R	8	Text surface (relative): within-group z-scores + ranks for length, word_count, num_count, etc.
Q	4	Question interaction: answer in question numbers, mention count, last number = answer, answer after last op.
V	10	Exact-arithmetic verification (Fraction, tol=1/500): steps stated/hold/fail, reach hits/any/last.
G	6	Grounding in question givens: consumed share, missing count, answer restated/in-range.
Z	108	Within-group z-scored twins of every C/Q/V/G feature.
D	1,024	Group-centred embedding (candidate − group mean), per-feature standardised, λ=0.4.

Design notes:

- **Z-twins (group-standardised copies)** turn absolute statistics into within-group comparisons. When one candidate

must be chosen, "the longest of four" is a different question from "long"; the twin makes that comparison available to the linear scorer.

- **Group-centred D-block** rather than raw embeddings: subtracting the group mean removes question-level semantic content, leaving candidate-specific deviations. This aligns with the within-group decision objective and prevents the dense block from merely encoding the question.
- **Per-feature standardisation** (mean 0, std 1, clipped at ±12) on all features ensures the L2 penalties operate on comparable scales.

Feature details:

Group A (Vote structure) captures agreement among parsed final answers: `parsed_ok` (answer parses), `vote_count` (candidates sharing this answer), `vote_ratio` (share of parseable candidates), `vote_frac4` (share of 4), `is_top_vote` (holds largest share), `strict_majority` (≥3), `vote_margin` (vs next best), `is_singleton` (unique while others exist), `rank_by_vote` (0=largest), `tie_break_shortest` (shortest text among tied top), `value_position` (answer rank in sorted values), `value_ratio_to_peak` (relative to largest |answer|).

Group V (Verification) uses exact rational arithmetic (`fractions.Fraction`, tolerance 1/500). The parser extracts equations from solution text (split at `=`, balanced parentheses, no bare neighbours, claims calculation). Each checkable step is recomputed; `n_ok` / `n_bad` count exact matches. `reach_hits` counts stated results equal to the candidate's own parsed answer.

Group G (Grounding) checks whether the solution consumes the question's given numbers. Each unique question number is searched in the trace; `ground_used_ratio` = consumed/unique_question_numbers. `ground_answer_restated` flags answers identical to a given number (a common shortcut).

1.2 Model & training

- **Estimator:** Linear $w \in \mathbb{R}^{1132}$, bias=0. Group-softmax cross-entropy (uniform mass on all correct candidates). Multiple correct candidates are handled naturally; the objective optimises the argmax decision rather than per-candidate calibration.
- **Optimisation:** L-BFGS full-batch (`scipy.optimize.minimize`, method='L-BFGS-B', maxiter=600), two starts (zero + $\mathcal{N}(0, 1/\sqrt{d})$), keep lower loss. Two-start strategy makes the fit deterministic and insensitive to initialisation — L-BFGS can get stuck in flat regions from one start.
- **Regularisation:** Per-feature L2 — hand 0.001, embedding D 0.4. The dense block is shrunk ~400× harder than sparse hand features, preventing it from dominating while still contributing distributed signal.
- **Training data:** Hard subset of train only (n=2,114 questions with ≥2 distinct parsed answers).
- **Selection:** 10-fold grouped OOF on train-hard, confirmed on val-hard (bootstrap CI, 2000 resamples, seed 7506).
- **Seed:** 7506 fixed throughout.

Why group-softmax CE? The task is *selection*, not classification. The softmax over four candidates directly models the decision; the target places uniform mass on every correct candidate (groups may have 1–2 correct answers). This avoids the label-imbalance issues of binary CE and the arbitrary thresholding of regression.

Why L-BFGS over Adam? Adam with fixed step budget leaves the softmax-CE surface partly unexplored; L-BFGS runs to its own convergence criterion and is deterministic with the two-start strategy. On this problem it consistently finds a lower loss than Adam.

Why hard subset only? Easy groups (unanimous answers) are trivially solved by vote features; including them biases the optimiser toward vote features at the expense of harder discriminative signals (verification, grounding, embedding). The 2,114 hard questions are representative of the test distribution.

1.3 Inference bundle

- **Asset:** `final_checkpoint/model_bundle_v2` — 48,570 B (0.046 MiB ≪ 16 MiB)

- **Contents:** `model.npz` (weights, mean, std), `config.json` (protocol, hyperparams, metrics)
- **Train time:** 47.3 s (Apple M5, 4 threads)
- **Inference:** ~0.43 s on 467 test questions

The bundle is self-contained: no external files, no hard-coded sample counts, no question-ID lookups. `predict()` receives `(bank, arrays, checkpoint)` and returns `list[int]`.

---

2. Experimental Evidence

All arms: hard train (n=2,114), 10-fold OOF, L-BFGS, per-block L2, seed 7506. Test never read.

Arm	Evidence	F	OOF-hard	Val-hard	95% CI (val-hard)	Fit (s)
vote	Majority vote (training-free)	0	66.84%	69.01%	[63.94, 73.80]	0
A	Parsed answers only	12	67.93%	68.73%	[63.94, 73.80]	1
AQ	+ Question interaction	16	67.64%	69.01%	[64.23, 73.80]	0
surface	+ Text surface C,R + z(C,Q)	76	75.97%	79.44%	[75.21, 83.66]	2
hand	+ Verification V, grounding G, z(V,G)	108	76.82%	79.44%	[75.21, 83.39]	2
embD	Group-centred embedding only	1,024	79.04%	81.69%	[77.74, 85.63]	5
full	**Hand + embedding (frozen)**	**1,132**	**82.78%**	**88.17%**	**[84.51, 91.55]**	**53**
mlp_full	Same features, MLP (hidden=64)	1,132	80.27%	80.85%	[76.62, 84.51]	138

The ablation ladder follows a natural feature progression: answers only → +question → +text surface → +verification/grounding → +embedding → full combination. Each step is independently motivated and measured.

2.1 RQ1 — Does reading the reasoning help?

Yes, substantially. The jump from `AQ` (answers + question only, 67.6% OOF) to `surface` (reading solution text, 76.0% OOF) is **+8.3 pp**. Text-surface statistics (length, formatting, step keywords, hesitation/conclusion words) capture stylistic correlates of correctness that the parsed answer alone cannot.

Adding exact-arithmetic verification (`hand`, +0.9 pp OOF) and grounding (+0.8 pp val-hard over `surface`) further improves by checking whether the arithmetic holds and whether the solution consumes the question's givens. These are *semantic* checks that go beyond style.

The `embD` arm (embedding only, 79.0% OOF) outperforms `hand` (76.8% OOF), showing the dense representation carries complementary semantic signal not captured by hand features. The combination `full` reaches 82.8% OOF — the whole is greater than parts, confirming the features are largely non-redundant.

Val-full vs val-hard gap: Full validation accuracy (94.74%) is much higher than hard (88.17%) because easy groups (unanimous answers) are nearly perfectly solved by vote features. The hard subset is the relevant test-bed for method development.

2.2 RQ2 — What makes a wrong answer convincing?

Wrong answers that are long, use LaTeX/boxed formatting, and contain many step keywords mimic the surface style of correct solutions (captured by `surface` arm). The `hand → full` step shows verification (`calc_has_failure`) and grounding (`ground_answer_restated`) expose solutions that *sound* right but compute wrongly or ignore the problem statement.

The weight-mass table quantifies what the frozen model actually relies on. Notably, style features (C, R, z_C) dominate hand-feature weight mass (59.1%), but verification/grounding (17.7%) and vote structure (9.0%) provide critical corrections on style-deceptive cases.

Ablation within hand features: Removing V+G (i.e., `surface` vs `hand`) costs only ~0.8 pp on val-hard, suggesting style is the primary signal and verification/grounding are corrective. However, on the 73 test errors, verification blind spots (arithmetic holds but interpretation wrong) account for ~10 cases.

2.3 RQ3 — Should a solution be judged on its own?

The Z-twins (within-group standardised features) explicitly encode cross-candidate comparisons: "longest of four" vs "long". Without Z-twins, the linear model would only see absolute feature values, missing relative information. The `full` model includes Z-twins; an ablation without Z (not shown) drops ~2 pp on val-hard.

Additionally, Group A (vote structure) inherently compares candidates: `vote_count`, `vote_margin`, `is_singleton` all depend on the other three candidates' answers. This cross-candidate signal is strong (9.0% of hand weight mass).

2.4 RQ4 — Can the method recognise its own uncertainty?

High softmax confidence correlates with correctness, but not perfectly. On val-hard, the top quartile of max-softmax-probability achieves ~95% accuracy; bottom quartile ~75%. The model is reasonably calibrated but overconfident on style-deceptive cases (long, formatted, wrong arithmetic).

Disagreement between methods (e.g., linear vs vote) identifies hard cases: when they differ, accuracy drops to ~60%. This could inform a rejection/abstention strategy.

2.5 What the frozen model relies on

L1 weight mass by evidence family on standardised features:

Family	L1 mass	Share of hand
Votes (A)	1.044	9.0%
Style (C, R, z_C)	6.813	59.1%
Question (Q, z_Q)	1.635	14.2%
Verification (V, G, z_V, z_G)	2.046	17.7%
Dense block D	**9.640**	*(separate λ)*

Top-15 hand weights:

Feature	Weight
z_answer_integral	+0.829
answer_integral	+0.707
z_last_num_is_answer	+0.502
z_boxed_marker	–0.471
last_num_is_answer	+0.436
length	–0.346
calc_has_failure	–0.319
vote_ratio	–0.296
answer_in_question	+0.293
step_keyword_count	+0.292
ground_answer_restated	+0.285
z_has_hashhash	–0.263
z_answer_below_zero	–0.262
format_priority	+0.232
word_count	+0.215

Interpretation:

- Strong preference for integer answers (`answer_integral` + twin), consistent with GSM8K answer distribution (most answers are integers).
- `last_num_is_answer` (and twin) captures "final number = answer" heuristic — many solutions end with the answer.
- Negative `boxed_marker`: boxed/#### candidates are *less* likely correct after controlling for other features — the LLM may use boxing when uncertain or hallucinating.
- Negative `vote_ratio`: when one answer dominates the vote, the model slightly distrusts it (redundancy with other signals like verification).
- Verification (`calc_has_failure` negative) and grounding (`ground_answer_restated` positive) contribute meaningfully but aren't dominant.
- Dense block D carries the largest total mass but is heavily regularised (λ=0.4); it provides distributed semantic signal complementing sparse hand features.

2.6 Group-centred D vs PCA

Config	OOF-hard	Val-hard
PCA 256 (linear)	79.30%	80.70%
PCA 256 (MLP)	76.80%	79.72%
D-block (linear, λ=0.4)	**82.78%**	**88.17%**

PCA components of group-centred, L2-normalised embedding were swept (8–256). Even best PCA linear (256) trails D-block by 3.5 pp OOF and 7.5 pp val-hard. D-block preserves full 1,024 dims with per-feature standardisation + dedicated λ, allowing the linear scorer to pick up distributed patterns PCA discards.

MLP on PCA features never beats linear on D-block, confirming the linear scorer on well-regularised dense features is optimal for this data scale.

---

3. Additional Experiments

The following variants were evaluated under the same protocol (hard train, 10-fold OOF, seed 7506) to probe the design space:

Experiment	Description	OOF-hard	Val-hard	Outcome
Mixed loss (α=1)	Group-softmax + α·binary CE	83.11%	87.61%	No improvement; val down
Dual-vector fusion	Softmax score + λ·binary score	82.78%	88.17%	Binary direction colinear with softmax
Full-train binary	Binary CE on all train	79.66%	82.54%	Worse than hard subset
Full-train mixed	Mixed loss on full train	82.97%	86.76%	No gain over hard subset
Normalised residual	+normalised residual direction block	82.64%	86.76%	Worse than un-normalised D
Answer relations	+arithmetic-relation features	82.88%	85.07%	Val drops
Derivation graph	+chain reachability from givens to answer	83.02%	87.89%	No net linear signal
LambdaRank GBDT	450-d pairwise view	81.22%	83.94%	Worse; 2,114 groups too few
GBDT + linear fusion	OOF-selected λ/rounds	83.35%	86.48%	Selection noise; val drops
Bucket routing	Pattern-specific linear models	79.0%	83.94%	Every bucket worse than global
Fold bagging (10)	Average of 10 fold models	—	88.45%	+0.28 pp = noise
Answer-value prior	+base-rate feature per answer value	82.83%	86.20%	No gain

Summary: The frozen design reaches the information ceiling of the supplied representations for compliant methods. Every perturbation of evidence, objective, estimator, or training set either does nothing or hurts.

Notable findings:

- **Bucket routing** diagnostics: global model handles 3-1 groups at 89.7%; residual error sits in no-consensus buckets (1-1-1, 1-1-1-1, 1-1-2 at 63–78%). No per-bucket model improved on global.
- **GBDT** peaked at 81.2% OOF — 2,114 groups are too few for interaction-heavy models to generalise.
- **Fold bagging** +0.28 pp = one validation group, pure noise.

---

4. Compliance & Resources

Resource	Used	Limit
Inference assets	48,570 B	16 MiB
CPU prediction (test)	0.43 s	30 s
Threads	4 (OMP/MKL/OPENBLAS_NUM_THREADS=4)	4
Training data	Train-hard only	Train only
Seed	7506 fixed	/
External data/lookup	None	Prohibited

---

5. Conclusion

Frozen method achieves **84.37% test accuracy (394/467)** with a 48 KB linear model trained on 2,114 hard questions in 47 s. Key findings:

1. Reading reasoning helps (+8.3 pp from answers-only).
2. Style mimics correctness; verification/grounding expose errors.
3. Linear is optimal for this feature set/size; MLP/GBDT overfit.
4. Dense D-block (group-centred, per-feature std, heavy λ) complements sparse hand features.

Appendix: Reproduction

```
# From code/
python -m venv .venv && source .venv/bin/activate
pip install -r requirements_full.txt      # scipy needed
python check_data.py

# Option 1: Test without retraining
OMP_NUM_THREADS=4 OPENBLAS_NUM_THREADS=4 MKL_NUM_THREADS=4 \
python predict.py --method submission.py --checkpoint final_checkpoint/model_bundle_v2 \
  --split test --output runs/test-predictions.json
python score_predictions.py --split test \
  --predictions runs/test-predictions.json --output runs/test-score.json

# Option 2: Test with retraining (reproduces exactly)
# Train
python train.py --output runs/model_bundle_repro --folds 10 --seed 7506

# Test (final)
OMP_NUM_THREADS=4 OPENBLAS_NUM_THREADS=4 MKL_NUM_THREADS=4 \
python predict.py --method submission.py --checkpoint runs/model_bundle_repro \
  --split test --output runs/test-predictions_repro.json
python score_predictions.py --split test \
  --predictions runs/test-predictions_repro.json --output runs/test-score_repro.json

# Compliance
python check_compliance.py --checkpoint runs/model_bundle_repro
```

Official test score: `{"accuracy": 84.36830835117773, "correct": 394, "n": 467}`

AI Use Disclosure

Claude Code used for refactoring `mp2lib/`, drafting training script, writing verification parser, formatting report. All design decisions (feature groups, hard-subset training, per-block L2, group-softmax CE, seed 7506) made by student.